# Development Setup Guide — GM Campus Power Quality System

Oct 28, 2026 · Gaurav Lokhande, Krishnapriya Nair, Hanna Pohl

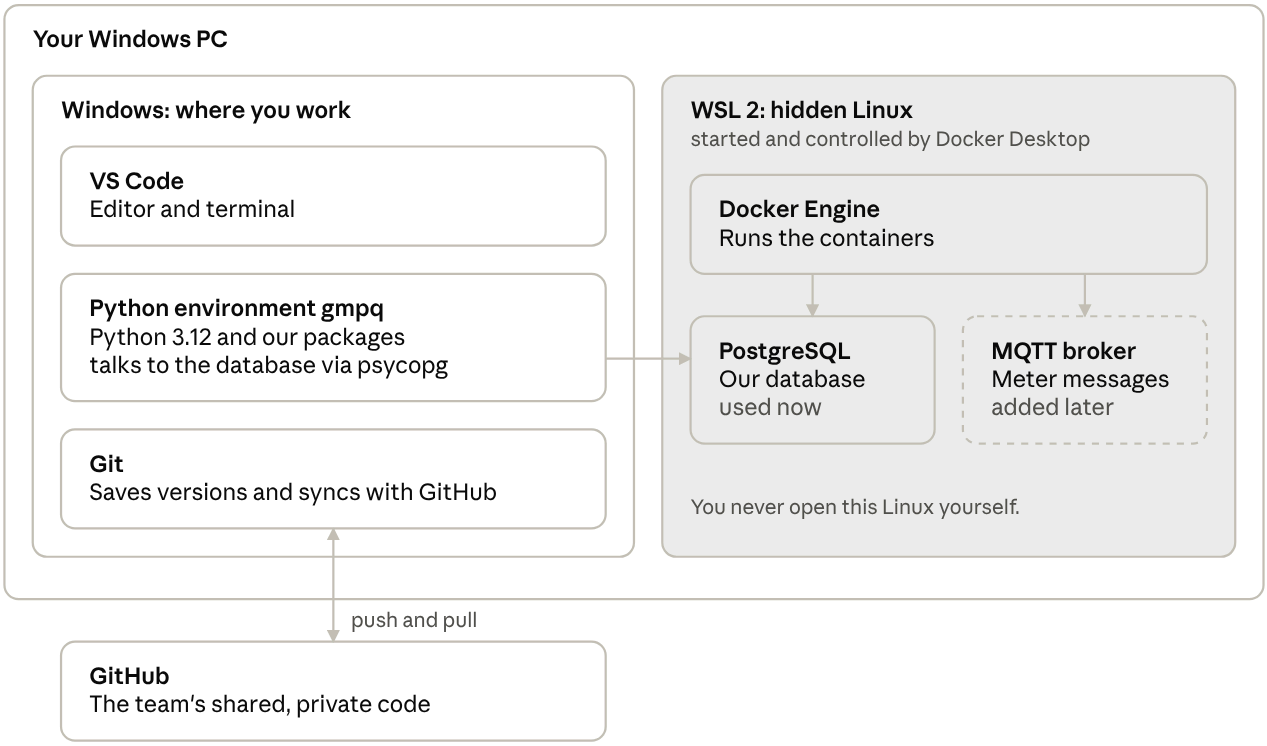
Every team member sets up the same tools on their own Windows PC, so the code runs the same way for all three of us. Plan about 45 minutes, plus download time.

## What we install and why

| Tool | What it is | Why we need it |
|---|---|---|
| Miniforge (conda) | Manages separate Python environments | Keeps this project's packages apart from other coursework |
| Python 3.12, in environment `gmpq` | The programming language | Everything we write: meter reading, the web app, reports, machine learning |
| Python packages | Ready-made libraries such as Django, pandas and scikit-learn | Listed in `requirements.txt`; installed with one command |
| VS Code | Code editor with a built-in terminal | Where we write and run the code |
| Git | Version control | Saves every version of the code and shares it with the team |
| GitHub | Online home for the code | Our shared, private repository |
| Docker Desktop | Runs software in containers | Runs the database now, and the whole system later, the same way on every PC and on the server |
| WSL 2 | A small Linux layer built into Windows | Docker needs it to run its containers |
| | | |

# How the parts fit on one PC

**Your code runs in Windows; Docker runs the database in a hidden Linux**




One PC · the Windows side where you work, and the hidden Linux side Docker uses

You work only on the Windows side. Docker Desktop starts the Linux side for you, and GitHub keeps the team's code in one place.

## Step 1: Project folder and VS Code

Keep the project **outside OneDrive**. OneDrive syncs files while programs are writing them, which breaks Git, Python environments and Docker. GitHub is our backup instead.

1. Open a terminal and create the folder (use your own Windows user name):

```
mkdir C:\Users\<your-name>\Projects\ITS_PM
```

2. In VS Code, click **File → Open Folder…**, select `ITS_PM` and click **Select Folder**. If asked whether you trust the authors, click **Yes**.
3. Open the terminal with **Terminal → New Terminal** (or **Ctrl + `**). The line should end in `…\Projects\ITS_PM>`.

4. In Extensions (**Ctrl + Shift + X**), install **Python** by Microsoft. Search for **Docker** and install Microsoft's container extension as well.

## Step 2: The project's own Python environment

The environment `gmpq` is a separate folder with its own Python and packages. It does not change your `base` environment or any other project.

In the VS Code terminal, run one command at a time:

```
conda create -n gmpq python=3.12 -y
conda activate gmpq
python --version
```

The line should now start with `(gmpq)` and show `Python 3.12.x`.

Then tell VS Code to use it: press **Ctrl + Shift + P**, type **Python: Select Interpreter**, and choose the entry with **gmpq**. VS Code remembers this for this folder only.

## Step 3: Python packages

All packages are listed in `requirements.txt` in the project folder, so everyone installs the same ones with one command.

| Group | Packages | Used for |
|---|---|---|
| Web application | Django 5.2 LTS, psycopg, python-dotenv, django-axes | The app, the database connection, keeping passwords out of code, locking accounts after failed logins |
| Data and dashboards | pandas, plotly | Working with measurements, interactive charts |
| Scheduled jobs | APScheduler | Health check every few minutes, report every Monday |
| Meters and messaging | paho-mqtt, pymodbus, asyncua | MQTT, and reading meters over Modbus TCP or OPC UA |
| Machine learning | scikit-learn, lightgbm | Clustering, anomaly detection, peak forecast |

| Group | Packages | Used for |
|---|---|---|
| Development tools | ipykernel, pip-audit | Notebooks in VS Code, checking packages for security holes |

### First person: install and save exact versions

Make sure the line starts with `(gmpq)`, then run:

```
python -m pip install -r requirements.txt --timeout 60 --retries 10
python -m django --version
python -m pip check
python -m pip list --format=freeze > requirements-lock.txt
```

You should see `5.2.18` and `No broken requirements found.` The last command writes the exact version of every package into `requirements-lock.txt`.

### Everyone else: install the exact same versions

```
python -m pip install -r requirements-lock.txt --timeout 60 --retries 10
```

This gives identical packages on all three PCs. `--timeout` and `--retries` only help on a slow connection; they change nothing in the files.

The database (PostgreSQL) is not a Python package. It runs in Docker (Step 4); `psycopg` is only the connector Python uses to talk to it.

## Step 4: Docker Desktop

### Why Docker needs Linux

The programs we run in Docker, such as the PostgreSQL database and later the MQTT broker, are packaged as small Linux systems called containers. They need a Linux engine underneath.

Windows has this built in: **WSL 2** runs a small, hidden Linux in the background, and Docker Desktop uses it automatically. You do not install Ubuntu or switch operating systems; you keep working in Windows and VS Code. The campus server will most likely run Linux too, so the system behaves the same there.

### 4a. Check your PC

1. Press the **Windows key**, type `winver` and press Enter. You need **Windows 11 version 23H2 or newer**, or **Windows 10 version 22H2**.
2. In the terminal, run `wsl --version`. If it shows `WSL version:` with **2.1.5 or newer** (for example 3.0.3), WSL is ready. If it says *The Windows Subsystem for Linux is not installed*, follow **4b. Install WSL** below.
3. Docker needs at least **8 GB of RAM**; 16 GB is comfortable.

### 4b. Install WSL (only if it is missing)

1. If the terminal shows *Press any key to install Windows Subsystem for Linux*, press **ESC**. We install it with administrator rights instead.
2. Right-click the **Start button** and choose **Terminal (Admin)** (on some PCs **Windows PowerShell (Admin)**). Click **Yes** when Windows asks for permission.
3. In that admin window, run:

```
wsl --update
```

4. Wait until it says *The operation completed successfully*. It downloads and installs WSL and switches on the **Virtual Machine Platform**, the Windows feature Docker needs.
5. **Restart the PC.** The message *Changes will not be effective until the system is rebooted* is normal; WSL only works after a restart.
6. After the restart, open a terminal and run `wsl --version`. The first line should show `WSL version: 3.0.3` or newer.

The version number of the WSL program (such as 3.0.3) is not the same as **WSL 2**, the technology Docker uses; any version from 2.1.5 upwards is fine. You do not need to install Ubuntu or any other Linux: Docker Desktop brings what it needs.

### 4c. Install Docker Desktop

1. Download Docker Desktop from **docker.com/products/docker-desktop**: choose **Download for Windows – AMD64** (ARM64 only for Snapdragon laptops).
2. Run **Docker Desktop Installer.exe** and keep **Use WSL 2 instead of Hyper-V** ticked. If this option does not appear (normal on Windows Home), that is fine: Docker then uses WSL 2 automatically. You can confirm later in Docker Desktop under Settings → General: Use the WSL 2 based engine is ticked.
3. Click **Close** when it finishes, then start **Docker Desktop** from the Start menu.

4. Accept the subscription terms. Docker Desktop is free for education and personal use; a Docker account is not needed.
5. Wait until the bottom-left corner shows **Engine running**.

### 4d. Check Docker works

Open a **new** terminal in VS Code, then run:

```
docker --version
docker compose version
docker run hello-world
```

The last command should print **Hello from Docker!**

This means Docker downloaded a ready-made package (an **image**) from Docker Hub and started it as a **container**, the same way we will start the PostgreSQL database later. You can also check in Docker Desktop under **Settings → Resources**: it should say *You are using the WSL 2 backend*.

Optional: in Docker Desktop under **Settings → General**, untick **Start Docker Desktop when you sign in**, and start it only when you work on the project.

## Sources

- Install Docker Desktop on Windows: system requirements, WSL 2 and install steps
- Docker Desktop license agreement: free for education and personal use
- Django download and supported versions: 5.2 LTS supported until April 2028